

SOMMAIRE

1	La Plate-Forme Microsoft .NET	3
1.1	Les composants de .NET	3
1.2	L'architecture .NET Framework	3
1.2.1	CLR	4
1.2.2	Les bibliothèques de classes .NET Framework	4
1.2.3	MSIL et les JITters	5
1.2.4	Système de types commun (CTS)	7
1.2.5	Métadonnées et Réflexion	7
1.2.6	Sécurité	7
2	Concepts et notions de bases	8
2.1	Architecture d'un projet C#	8
2.2	Le système de types	11
2.2.1	Valeurs et références	11
2.2.2	System.Object	12
2.2.3	Types et Alias	13
2.2.4	Espaces de noms (namespace)	14
2.3	Le langage de programmation C#	15
2.3.1	Types et variables	15
2.3.2	Expressions et opérateurs	17
2.3.3	Contrôle de flux	19
2.3.4	Procédures et fonctions	20
2.3.5	Gestion des exceptions	21
2.4	Classes	22
2.4.1	Définition d'une classe	22
2.4.2	Héritage	23
2.4.3	Les méthodes	24
2.4.4	Le polymorphisme	27
2.4.5	Surcharge	31
2.4.6	Les membres simples	32
2.5	Interfaces	33
2.6	Délégués et gestionnaires d'événements	34
3	Contrôles et Objets de base	34
3.1	Contrôles et objets usuels	35
3.1.1	Contrôles Label, TextBox, Button, radioButton, ListBox, ComboBox	35
3.1.2	Les paramètres des événements	36
3.1.3	L'objet MessageBox	37
3.1.4	Le contrôle Timer	37
3.1.5	Le contrôle PictureBox	37
3.1.6	Contrôle ImageList	38
3.1.7	Les contrôles ListView et TreeView	38

3.1.8	Les contrôles DomainUpDown et NumericUpDown	39
3.1.9	Le contrôle DateTimePicker	40
3.2	Quelques objets spécifiques.....	40
3.2.1	Les contrôles OpenFileDialog et SaveFileDialog.....	40
3.2.2	Le contrôle RichTextBox	41
3.2.3	L'objet Menu	42
3.2.4	Applications MDI.....	42
4	Exercices.....	43

1 La Plate-Forme Microsoft .NET

.NET représente un univers où les entités coopèrent pour fournir des solutions aux utilisateurs. .NET comprend quatre grands composants.

1.1 Les composants de .NET

- .NET Building Block Services : permet d'accéder par programme à certains services, tels que gestion de fichiers, calendrier ou Passport.NET (contrôle d'identité).
- .NET device software : exécuté sur les nouveaux équipements Internet
- .NET user experience : regroupe des fonctionnalités telles qu'une interface naturelle, des agents d'information et balises intelligentes (smart tags).
- .NET infrastructure : regroupe l'architecture .NET Framework, l'environnement de développement Visual Studio .NET, les serveurs .NET Entreprise et Windows .NET

1.2 L'architecture .NET Framework

.NET infrastructure regroupe toutes les technologies constituant le nouvel environnement de création-exécution d'applications robustes, évolutives (« scalable ») et distribuées. La partie de .NET permettant aux développeurs de créer ces applications, c'est l'architecture .NET Framework.

Le .NET Framework est constitué d'un ensemble d'outils révolutionnaires. Ceux-ci sont organisés sous forme de classes, d'assemblies et d'espaces de noms.

- Un assembly est représenté par un fichier bibliothèque .dll contenant des classes.
- Un espace de noms peut être contenu dans un ou plusieurs assemblies, il regroupe des classes du même type.
- Une classe définit les caractéristiques d'un objet. Une classe est accessible par l'intermédiaire de l'espace de noms dans lequel elle est incluse.

Le .NET Framework est orienté objet. Il fournit aux développeurs de nombreuses classes et espaces de nom exploitables par un programme, y compris par une application ASP.NET. Dans ce système, chaque classe hérite des fonctionnalités de la classe de laquelle elle dérive. L'architecture .NET peut être représentée sous forme d'arbre structuré à la manière d'un arbre

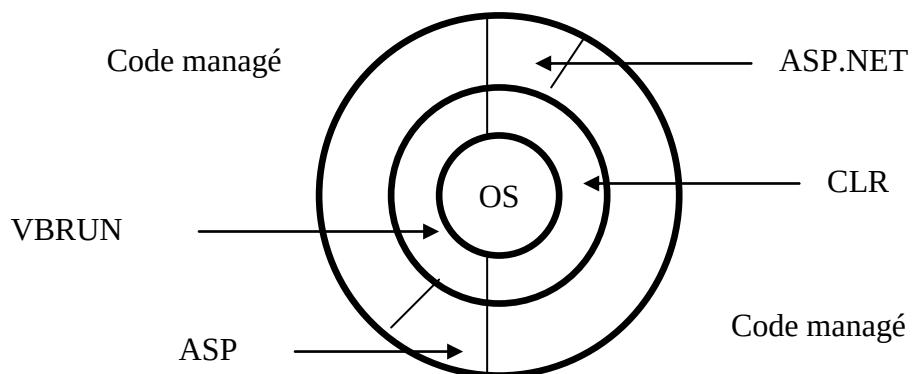
généalogique où au sommet la classe Object fournit des propriétés de base à toutes les classes.

1.2.1 CLR

Le .NET Framework contient le CLR (Common Language Runtime), un système permettant à des langages de différentes origines de tirer de ses fonctionnalités. Le CLR assure une compatibilité à tous les langages (C++, C#, J#, Visual Basic et autres non microsoft) qui l'exploitent, on parle d'interopérabilité « interlangage » (cross-language). Le support multilangage permet de programmer dans son langage préféré, ensuite le compilateur spécifique à votre langage rend compatible le programme dans le .NET Framework.

Cette interopérabilité s'appuie sur CLS (Common Language Specification), un ensemble de règles que doit respecter tout compilateur désireux de créer des applications .NET exécutables sous CLR.

On parle à ici de code géré ou managé (managed code), dans les sens que le code exécuté est sous le contrôle de CLR. Dans un environnement de code géré, un certain nombre de règles garantissent que les applications se comportent d'une manière globalement uniforme, et ce, indépendamment du langage ayant servi à les écrire. Le comportement homogène des applications est l'essence de .NET. Ces règles globales avant tout les créateurs de compilateurs.

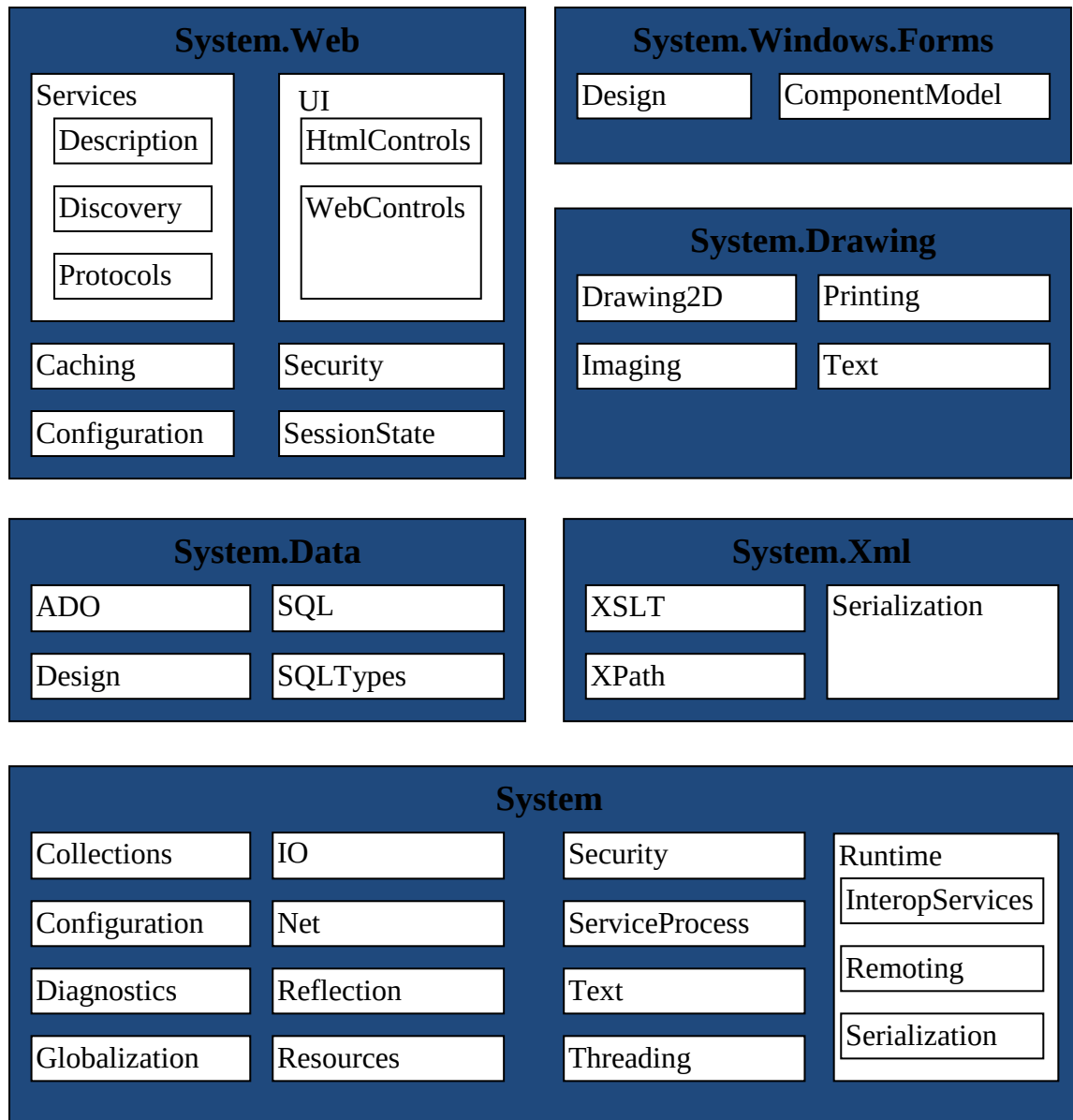


Le CLR et ses relations avec d'autres environnements d'exécution

1.2.2 Les bibliothèques de classes .NET Framework

Les bibliothèques de classes sont extrêmement importantes pour l'interopérabilité entre les langages, car elles permettent aux développeurs d'utiliser une même interface de programmation pour toutes les fonctionnalités

exposées par CLR. Les langages ne sont que des interfaces syntaxiques vers les bibliothèques de classes.



1.2.3 MSIL et les JITters

Pour permettre aux créateurs de langages de porter ces derniers vers .NET, Microsoft a inventé une sorte d'assembleur baptisé MSIL (Microsoft Intermediate Language). Pour compiler une application destinée à .NET, le compilateur concerné lit le code source (écrit en VB, C# ou tout autre langage conforme aux spécifications CLS), puis génère du code MSIL. Ce code MSIL sera traduit en langage machine lorsque l'application sera exécutée pour la première fois par CLR. Plus précisément voici l'enchaînement des opérations :

1. Vous écrivez votre code source en VB ou C#.
2. Vous compilez le code source à l'aide du compilateur VB ou C# pour créer un EXE.
Le compilateur écrit le code MSIL, plus un manifeste dans une partie accessible en lecture seule de l'EXE ayant un entête standard PE (Win32-Portable Executable). Le point clé est le suivant : le compilateur importe aussi depuis l'environnement d'exécution .NET la fonction `_CorExeMain`.
3. Lors de l'exécution de l'application, le système d'exploitation charge l'en-tête PE ainsi que toutes les DLL dépendantes, par exemple celle qui exporte `_CorExeMain` (mscorlib.dll), comme il le fait d'ailleurs pour chaque PE valide.
Le chargeur du système d'exploitation saute ensuite au point d'entrée situé dans le PE et placé par le compilateur. C'est ainsi que se passe l'exécution d'un PE sous Windows. Comme le système d'exploitation ne sait pas exécuter le code MSIL, le point d'entrée est un stub qui renvoie la fonction `_CorExeMain` de mscorlib.dll
4. La fonction `_CorExeMain` commence l'exécution du code MSIL placé dans le PE.
5. CLR traduit le code MSIL en langage machine à l'aide d'un compilateur just-in-time appelé aussi JITter. Le code ainsi produit est placé dans un cache, et il y a recompilation qu'en cas de modification du code source.

Il existe trois JITters différents :

- **Génération de code lors de l'installation** : il y a traduction en code machine d'un assemblage complet, comme le ferait un compilateur. Un assemblage est le paquet de code qui est envoyé au compilateur. Cette compilation se fait lors de l'installation. L'avantage est que la compilation de l'assemblage tout entier se fait une bonne fois pour toutes, juste avant l'exécution. On choisit cette option en fonction de l'application et de l'environnement de déploiement.
- **JIT** : il y a appel du JITter selon les modalités décrites dans l'énumération en amont, chaque fois qu'une méthode est appelée pour la première fois. C'est l'option par défaut.
- **EconoJIT** : option intéressante pour les systèmes ayant des ressources limitées, par exemple les équipements portables. Le JITter peut expulser le code compilé de la mémoire en cas de besoin. Mais si le code précédemment déchargé est appelé de nouveau, il faut le recompiler.

1.2.4 Système de types commun (CTS)

L'environnement d'exécution .NET permet aux développeurs de créer de nouveaux types, fonctionnellement semblables aux types prédéfinis. Tous les types, tant prédéfinis que personnalisés, s'utilisent d'une façon homogène, et ce, quel que soit le langage (conforme à la norme CLS) employé.

1.2.5 Métadonnées et Réflexion

Outre le fait qu'il traduit du code source en code MSIL, un compilateur compatible avec CLS effectue une autre tâche très importante : il incorpore des métadonnées à l'EXE résultant.

Ces métadonnées qui dérivent des éléments programmatiques constituant l'EXE (types déclarés, méthodes implémentées, etc.) ressemblent, sur le plan conceptuel, aux bibliothèques de types des composants COM. Les métadonnées produites par un compilateur .NET sont beaucoup plus complètes que les bibliothèques de type COM. En outre, elles sont systématiquement incorporées à l'EXE.

Les métadonnées permettent à l'environnement d'exécution .NET de savoir, au moment de l'exécution, quels sont les types qui seront alloués et quelles sont les méthodes qui seront appelées. Ainsi, l'environnement peut organiser son cadre de travail de la manière la plus efficace possible pour l'exécution de l'application. La lecture des métadonnées s'appelle la « réflexion ». les bibliothèques de classes .NET Framework apportent toutes sortes de méthodes de réflexion permettant à une application quelconque (et pas uniquement au CLR) de lire les métadonnées d'une autre application.

1.2.6 Sécurité

La sécurité, facteur crucial dans le contexte des applications distribuées, commence dès que CLR charge une classe. Au chargement d'une classe dans l'environnement d'exécution .NET, il y a vérification d'un certain nombre d'éléments liés à la sécurité : règles d'accès, contraintes d'autocohérence, etc. des contrôles de sécurité garantissent, en outre, que telle ou telle portion de code est habilitée à accéder à telle ou telle ressource. Pour cela, le code de sécurité vérifie rôles et données d'identification. Ces contrôles de sécurité franchissent même les frontières de processus et de machines, afin d'assurer la sécurité des données sensibles dans un contexte distribué.

2 Concepts et notions de bases

2.1 Architecture d'un projet C#

La notion de projet est à la base de tout développement en Visual Basic ou C# avec l'environnement Visual Studio .NET. Un projet permet de créer des applications, des composants et des services et englobe le code source de l'application ainsi que divers autres éléments essentiels au bon fonctionnement de l'ensemble :

- Des définitions de connecteurs de données ;
- Des pages HTML ;
- Des fichiers de classes ;
- Des services Web ;
- Des pages ASP ;
- Des feuilles de style ;
- Des références à des éléments externes...

Tous ces éléments sont optionnels.

Un projet constitue un conteneur, lui-même élément d'un autre conteneur : une solution (une solution peut contenir plusieurs projets dépendant les uns des autres). Lorsqu'on crée un projet, Visual Studio l'intègre automatiquement à une solution.

Un projet contiendra une multitude de fichiers aux extensions diverses en fonction des éléments à incorporer dans le projet. Les éléments suivants peuvent être incorporés à un projet :

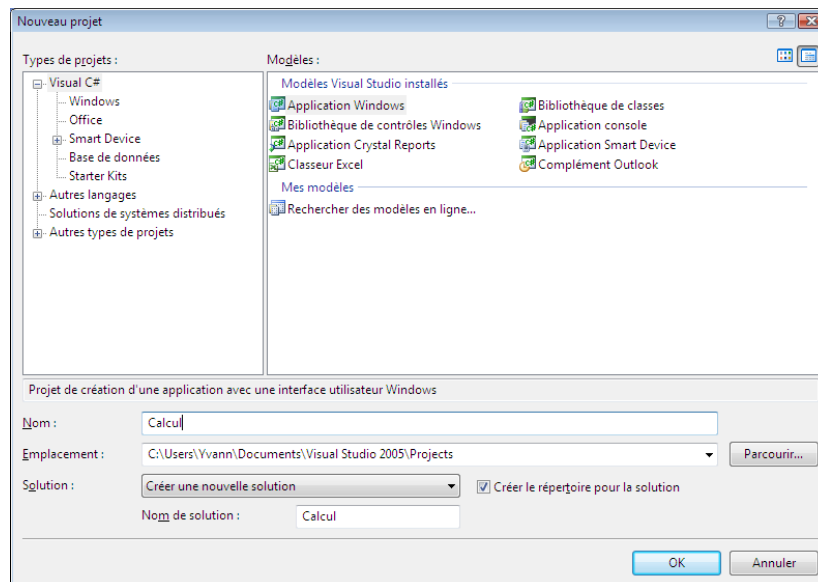
Élément de projet	Type de projet	Extension de fichier	Définition
Formulaire Windows	Local	.vb ou .cs	Élément définissant une fenêtre d'application Windows classique
Formulaire web	Web	.aspx	Élément définissant un formulaire web avec possibilité de disposition précise des zones
Classe	Local	.vb ou .cs	Élément contenant le code de déclaration d'une classe
Module de classe	Local	.vb ou .cs	Élément contenant le code de diverses fonctions et procédures
Contrôle utilisateur	Local	.vb ou .cs	Contrôle (élément visuel) que l'on peut utiliser sur un formulaire
Fichier de code	Local	.vb ou .cs	Fichier de code Visual Basic vide

	ou Web		
Page HTML	Local ou Web	.htm	Page web classique
Formulaire hérité	Local	.vb ou .cs	Formulaire dont l'aspect et le comportement sont issus de ceux d'un formulaire Windows existant
Fichier texte	Local ou Web	.txt	Fichier texte vide
Feuille de style	Local ou Web	.css	Feuille de style en cascade exploitée par des pages web HTML pour la définition des styles
Classe Installer	Local ou Web	.vb ou .cs	Fichier utilisé pour créer une procédure d'installation personnalisée
Etat Crystal Reports	Local ou Web	.rpt	Fichier destiné à la création d'un état Crystal Report ; l'ouverture d'un tel fichier provoque celle de l'application Crystal Report Designer
Bitmap	Local ou Web	.bmp	Image bitmap
Fichier curseur	Local	.vb ou .cs	Fichier de création d'un curseur personnalisé
Fichier icône	Local	.vb ou .cs	Fichier de création d'une icône personnalisée
Fichier JScript	Local ou Web	.js	Fichier de code JScript.NET vide
Fichier VBScript	Local ou Web	.vbs	Fichier de code VBScript vide (Visual Basic Scripting Edition)
Windows Script Host	Local ou Web	.wsf	Fichier de code vide utilisé pour les scripts
Page ASP	Web	.asp	Fichier ASP (Active Server Page) contenant des balises de formatage HTML ainsi que du code s'exécutant au niveau du serveur web
Fichier de	Local	.resx	Fichier permettant de stocker des

ressources	ou Web		informations propres à la langue cible du système (espagnol, français, etc.)
------------	-----------	--	---

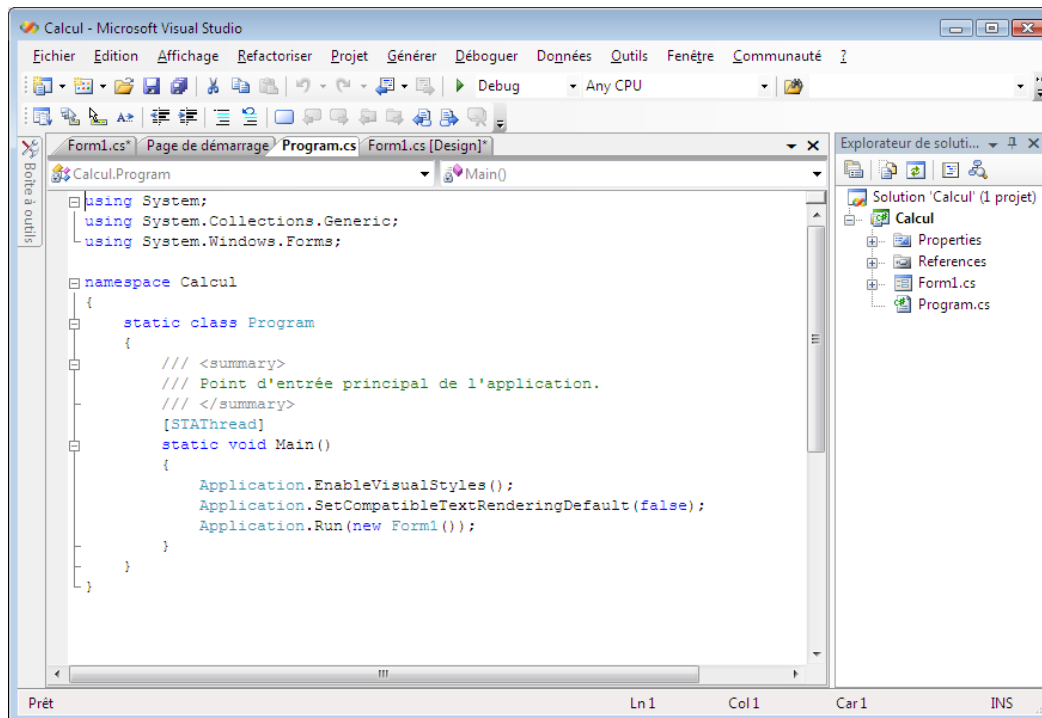
La liste des éléments n'est pas exhaustive.

Lors de la création d'un projet, l'on a le choix entre un ensemble de modèles de projets disponibles :



Modèle de projet	Utilisation
Application Windows	Création d'une application Visual Basic classique
Bibliothèque de classes	Création de classes réutilisables dans différents projets
Bibliothèque de contrôles Windows	Création de contrôles personnalisés utilisés dans les formulaires Windows
Application Web ASP.NET	Création d'une application Web ASP.NET intégrant des pages dynamiques
Service Web ASP.NET	Création de services web ASP.NET
Bibliothèque de contrôles Web	Création de contrôles personnalisés dédiés aux formulaires web
Application console	Création d'une application ligne de commande (type MS-DOS)
Service Windows	Création de services Windows, c'est-à-dire des applications ne nécessitant pas d'interface utilisateur
Nouveau projet dans un dossier existant	Création d'un projet vide dans un dossier existant afin de permettre la réutilisation des fichiers du projet existant
Projet vide	Création d'un projet vide... ou tout reste à faire !
Projet Web vide	Création d'un projet web vide

En C# un projet est constitué d'un ensemble de fichiers :



Entre autre un fichier programme (program.cs) qui sert de démarrage à l'application, et autant de fichiers qu'il y a de formulaires. Des fichiers de propriétés et des références à des classes sont automatiquement ajoutés.

2.2 Le système de types

En C# tout est objet ; le distinguo entre types prédéfinis, à savoir caractères, chaînes et nombres et les types personnalisés (ou « classes ») n'existe plus contrairement aux autres langages objet (non .NET). Dans le système de type commun (CTS) tout est objet et dérive implicitement d'une même classe System.Object.

2.2.1 Valeurs et références

Bien que le CTS soit entièrement orienté objet on distingue deux catégories de types : les types valeur et les types référence.

- Types valeur

Une variable de type valeur contient des données et ne peut prendre la valeur null. Par exemple `int i = 24` ; crée une variable ayant le type CTS System.Int32

donc il y a allocation de 32 bits sur la pile ; l'affectation entraîne le transfert vers cet espace alloué, de la valeur 24 stockée sur 32 bits.

Toute déclaration de variable de type valeur (type primitif, énumération ou structure) alloue sur la pile un nombre d'octets associés au type concerné et on manipule directement les bits alloués.

- Types référence

Un type référence ressemble à une référence du C++, en ce sens qu'il s'agit d'un pointeur « sécurisé au niveau du type ». Une référence si elle est différente de null pointe toujours vers un objet du type spécifié, objet qui a déjà été alloué sur le tas (heap). Par exemple `string s = "exemple"` ; une valeur est allouée sur le tas et `s` est un type référence (string) vers cette valeur.

Quand on déclare une variable C# de type référence (classe, tableau, délégué ou interface), on alloue sur le tas le nombre d'octets associés au type concerné et on manipule l'objet indirectement via une référence.

- Boxing et Unboxing

Le « boxing » est la conversion d'un type valeur en un type référence ; par exemple :

```
int i = 42 ;                //type valeur
object unobjet = i ;        //boxing de i pour donner un objet
```

La conversion en sens inverse s'appelle « unboxing » ; par exemple :

```
int i = 42 ;                //type valeur
object unobjet = i ;        //boxing de i pour donner un objet
int i2 = (int) unobjet ;    //unboxing redonne int
```

Le boxing ne demande aucune conversion explicite ce qui n'est pas le cas du unboxing.

2.2.2 System.Object

Comme tous les types dérivent d'une façon ou d'une autre de `System.Object`, ils ont en commun un certain nombre de méthodes.

Quatre méthodes publiques.

Méthode	Description
<code>bool Equals()</code>	Compare deux références d'objet ou types valeur pour savoir s'il s'agit du même objet ou les deux valeurs sont

	identiques
int GetHashCode()	Donne le code de hachage spécifié pour un objet
Type GetType()	Permet d'obtenir les données de type d'un objet
string ToString	Par défaut donne le nom de l'objet

Deux méthodes protégées : ***void Finalize()*** et ***Object MemberwiseClone***

2.2.3 Types et Alias

CTS définit des types susceptibles de resservir dans tous les langages .NET. Chaque langage définit des alias pour ces types. Les alias pour C# sont :

Type CTS	Alias C#	Description
System.Object	object	Classe de base de tous les types CTS
System.String	string	Chaîne de caractères
System.SByte	sbyte	Octet signé sur 8 bits
System.Byte	byte	Octet non signé sur 8 bits
System.Int16	short	Valeur signée sur 16 bits
System.UInt16	ushort	Valeur non signée sur 16 bits
System.Int32	int	Valeur signée sur 32 bits
System.UInt32	uint	Valeur non signée sur 32 bits
System.Int64	long	Valeur signée sur 64 bits
System.UInt64	ulong	Valeur non signée sur 64 bits
System.Char	char	Caractère Unicode sur 16 bits
System.Single	float	Valeur en virgule flottante IEEE sur 32 bits
System.Double	double	Valeur en virgule flottante IEEE sur 64 bits
System.Boolean	bool	Valeur booléenne (True ou False)
System.Decimal	decimal	Valeur sur 128 bits

- Conversion de type

La conversion entre les types simples char, int et float est implicite.

La conversion entre une chaîne et les autres types est définie par :

nombre → chaîne	<i>nombre.ToString()</i> ;
chaîne → int	<i>int.Parse(chaine)</i> ou <i>System.Int32.Parse(chaine)</i>
chaîne → long	<i>long.Parse(chaine)</i> ou <i>System.Int64.Parse(chaine)</i>
chaîne → double	<i>double.Parse(chaine)</i> ou <i>System.Double.Parse(chaine)</i>
chaîne → float	<i>float.Parse(chaine)</i> ou <i>System.Float.Parse(chaine)</i>

- Conversion entre Classe de base et Classe dérivée

Entre une classe de base et une classe dérivée, il peut y avoir conversion implicite vers l'amont (upcasting). Par exemple :

```
class Etudiant { }
class MaitriseEtudiant : { }
class Exemple1 {
    public static void Main () {
        Etudiant e = new MaitriseEtudiant() ;
    }
}
```

En revanche, la conversion implicite vers l'aval (downcasting) n'est pas possible ; il faut une conversion explicite. Par exemple :

```
class Etudiant { }
class MaitriseEtudiant : { }
class Exemple2 {
    public static void Main () {
        MaitriseEtudiant me = (MaitriseEtudiant) new Etudiant() ;
    }
}
```

2.2.4 Espaces de noms (namespace)

Les espaces de noms permettent d'organiser les classes et les autres types au sein d'une seule et même structure hiérarchique. Un même espace de noms peut s'étaler sur plusieurs fichiers de code source. Cela permet, par exemple, de placer dans un fichier source séparé chacune des classes d'un espace de noms. Ils sont définis à l'aide de l'instruction **namespace** et peuvent être imbriqués.

```
namespace <nom_espace>
{
    class <nomclasse>
    { ... }
    class <nomclasse2>
    { ... }
}
```

Ils offrent un moyen d'abréger des noms très longs de classes et autres types. Le mot clé **using** permet d'accéder à toutes les classes d'un espace de noms. Par exemple :

//utilisation du nom complet d'une classe

```
class Expl1 {  
    public static void Main() {  
        System.Console.WriteLine("test") ;  
    }  
}
```

```
//utilisation de l'espace de nom System  
using System ;  
class Expl2 {  
    public static void Main() {  
        Console.WriteLine("test") ;  
    }  
}
```

2.3 Le langage de programmation C#

La syntaxe du langage C# est très proche de celle du langage C/C++.
Son compilateur de ligne de commande est csc.exe et s'utilise de la manière suivante : **csc <nom_fichier_cs>**

2.3.1 Types et variables

La syntaxe de déclaration de variables est :

[portée] <type> <liste des variables> ;

Pour une constante on a : **[portée] const <Type> <identificateur> = <valeur> ;**

- Les tableaux

En C#, un tableau est un objet ayant pour classe de base **System.Array**.

Déclaration de tableau

<type> []<nomtableau> ; pour tableau à une dimension

<type> [...]<nomtableau> ; pour tableau à plusieurs dimensions

Par exemple :

int [, ,] entiers ; //déclare un tableau à 3 dimensions

On utilise le mot clé new pour créer un objet tableau par :

nomtableau = new type [taille] ;

Par exemple :

entiers = new int[2,4,3] ; //instancie un tableau multidimensionnel

Tableaux imbriqués (tableau de tableau) ou irréguliers :

<type> [] [] <nomtableau> ;

Par exemple :

```
using System ;
class Controle {
    virtual public void Salut() {
        Console.WriteLine("Contrôle");
    }
}
class Bouton : Controle {
    override public void Salut() {
        Console.WriteLine("Bouton");
    }
}
class Liste : Controle {
    override public void Salut() {
        Console.WriteLine("Liste");
    }
}
class AppTableauImbri {
    public static void Main() {
        Controle [][] controles;
        Controles = new Controle[2][];
        Controles[0] = new Controle[3];
        for (int i = 0; i < controles[0].Length; i++) {
            controles[0][i] = new Bouton();
        }
        controles[1] = new Controle[2];
        for (int i=0; i<controles[1].Length; i++) {
            controles[1][i] = new Liste();
        }
        for (int i =0; i < controles.Length; i++) {
            for (int j=0; j<controles[i].Length;j++)
            {
                Controle uncontrôle = controles[i][j];
                Uncontrôle.Salut();
            }
        }
        string str = Console.ReadLine();
    }
}
```


Le tableau imbriqué contient un tableau de 3 boutons et un tableau de 2 listes.

- Structures et énumérations

Déclaration d'un type de structure :

```
struct <type> <identificateur>{
    <type1><liste1 de champs> ;
    <type2><liste2 de champs> ;
    ...
}
```

Déclaration d'un type énumération :

```
enum <type> <identificateur>{
    valeur1,
    valeur2,
    ...
}
```

2.3.2 Expressions et opérateurs

Les opérateurs de C# sont quasi identiques à ceux du langage C/C++. Le tableau suivant récapitule les opérateurs par ordre de priorité.

Catégories	Opérateurs
Opérateurs fondamentaux	(x), x.y, f(x), a[x], x++, x--, new, typeof, sizeof, checked, unchecked
Opérateurs unaires	+, -, !, ~, ++x, --x, (T)x
Opérateurs multiplicatifs	*, /, %
Opérateurs additifs	+, -
Opérateurs de décalage	<<, >>
Opérateurs de comparaison	<, >, <=, >=, is
Opérateurs d'égalité	==
Opérateurs logiques AND, XOR, OR	&, ^,
Opérateurs conditionnels AND, OR	&&,
Opérateur conditionnel	?:
Opérateur d'affectation	=, *=, /=, %=, +=, -=, <<=, >>=, &=, ^=, =

- Opérateur **typeof**

L'opérateur **typeof** appliqué à une classe renvoie un objet de **System.Type** dont

les méthodes **GetMethods** et **GetMembers** retournent respectivement les méthodes et les membres de cet objet. Par exemple soit la classe Essai suivante :

```
public class Essai{
    public int num ;
    public void rien() { }
}
```

Type t = typeof(Essai) ; renverrait un objet de **System.Type** de sorte que **t.GetMethods()** retourne l'ensemble des méthodes de la classe Essai et **t.GetMembers()** retourne l'ensemble des membres.

Si une instance de la classe existe, on obtient l'objet type en appelant la méthode **GetType()**. Par exemple,

```
Essai essai1 = new Essai() ;
Type t1 = essai1.GetType() renverrait le même objet que t
```

- Opérateurs **is** et **as**

L'opérateur **is** permet de savoir si un type est compatible avec un autre ; la syntaxe est : **expression is type**, par exemple :

```
Classe classe1 = new Classe() ;
If (classe1 is ClasseX)
{ ....
}
```

L'utilisation la plus courante de cet opérateur consiste à déterminer si un objet prend en charge une interface particulière.

L'opérateur **as** est semblable à l'opérateur **is** , sauf qu'en plus il effectue la conversion. La syntaxe est : **objet = expression as type** ;

- Surcharge d'opérateurs

Un certain nombre d'opérateurs sont susceptibles d'être surchargés. Ce sont :

- Opérateurs unaires +, -, !, ~, ++, --, true, false ;
- Opérateurs binaires +, -, *, /, %, &, |, ^, <<, >>, ==, !=, >, <, >= et <=

La syntaxe de surcharge est :

```
public static <type> operator <symbol_opérateur> (objet1 [, objet2])
{ corps }
```

Par exemple :

```
struct complexe { float reel, imag ;} ;
public static complexe operator + (complexe x, complexe y)
```

```
{ complexe z ;  
    z.reel = x.reel + y.reel ;  
    z.imag = x.imag + y.imag;  
}
```

2.3.3 Contrôle de flux

2.3.3.1 Structures conditionnelles

- Structure *if*

La syntaxe générale est :

```
if(expression)  
    instruction1  
[else  
    Instruction2]
```

- Structure *switch*

La syntaxe générale est :

```
switch(expression) {  
    case expression-constante : instruction ; instruction-de-saut ;  
    ...  
    case expression-constanteN : instructionN ; instruction-de-saut ;  
    [default : instruction ; instruction-de-saut]  
}
```

2.3.3.2 Structures itératives

- Structure *while*
while(expression)
 corps

- Structure *do/while*
 do
 corps
 while (expression)

- Structure *for*

 for (initialisation ; expression ; pas)
 corps

Les parenthèses du **for** comprennent trois parties, dont chacune peut être vide. Dans les parties « initialisation » et « pas » de la structure **for**, l'opérateur virgule permet de spécifier plusieurs instructions qui seront exécutées en séquence. La partie « initialisation » est effectuée une seule fois, avant la première exécution de la boucle.

- Structure **foreach**

A l'instar de Visual Basic, C# offre une structure permettant de parcourir tableaux et collections. La syntaxe est :

foreach (variable in expression)
corps

2.3.3.3 Débranchements

- Instruction **break**

Permet de sortir d'une boucle ou d'une structure conditionnelle.

- Instruction **continue**

L'instruction **continue** arrête l'exécution du corps de la boucle et fait sauter le programme au début du passage suivant dans la boucle.

- Instruction **goto**

Les syntaxes possibles sont :

goto identificateur
goto case expression-constante
goto default

Dans la première variante, le programme continue vers l'instruction repérée par l'étiquette:

identificateur :

Les deux dernières variantes s'utilisent dans un **switch**.

- Instruction **return**

Spécifie une valeur à retourner à l'appelant (si le code appelé n'est pas étiqueté **void**) et provoque l'arrêt de l'exécution du code appelé. La syntaxe est :

return [expression]

2.3.4 Procédures et fonctions

La syntaxe de déclaration d'une procédure ou fonction est la suivante :

```
<type> <nom-sous-programme> (<liste des paramètres>)  
    {  
        corps  
    }
```

Pour une procédure le type est nécessairement **void** et le corps ne doit pas contenir l'instruction **return**.

Pour une fonction le type doit être spécifié et une instruction « **return expression** » doit figurer dans le corps.

2.3.5 Gestion des exceptions

Le traitement des exceptions repose sur 4 mots clés : **try** – **catch** – **throw** et **finally**.

- Déclenchement d'exception

Quand une méthode se trouve devant une situation exceptionnelle, elle déclenche une exception qu'elle passe à la méthode appelante via **throw**. Le mot clé **throw** crée un objet de type **System.Exception** ou d'un type dérivé.

```
public void MaMethode() {  
    //détection d'une erreur  
    throw new Exception() ;  
}
```

- Interception d'une exception

Si une méthode peut créer une exception, une autre peut intercepter cette exception. Le mot clé **catch** définit un bloc de code, dit « gestionnaire d'exception », qui sera exécuté quand une exception d'un certain type aura été interceptée. Pour intercepter une exception, on place dans un bloc **try** le code à exécuter puis on spécifie dans un bloc **catch** les types d'exception que le code du bloc **try** doit traiter.

La syntaxe est :

```
try  
{    le code protégé  
}  
Catch(Exception e)  
{    code de gestion des exceptions
```

```
}
```

On peut ajouter un bloc de code qui sera exécuté dans tous les cas qu'il y ait interception ou non à l'aide du mot clé **finally**.

```
finally  
{ code exécuté en fin de compte  
}
```

2.4 Classes

2.4.1 Définition d'une classe

La syntaxe de déclaration d'une classe est :

```
<modificateur> class <Nomclasse> {  
    <modificateur> <type> <nom_membre_simple> ;  
    ...  
    <modificateur> <type> <nom_méthode>(<liste paramètres>  
        { corps méthode } ;  
    ...  
}
```

Une classe peut disposer des différents membres suivants :

- Champs : variables membres contenant des valeurs, qui ne sont autres que les données de l'objet.
- Méthodes : désignent le code opérant sur les données (champs) de l'objet.
- Propriété : une propriété est une méthode qui donne aux clients de la classe l'impression que c'est un champ.
- Constante : une constante est un champ contenant une valeur qui est impossible de modifier
- Indexateur : est un membre qui permet d'indexer un objet via des méthodes d'accès get et set.
- Événement : est une action, par exemple un clic sur une fenêtre, qui provoque l'exécution d'une portion du code.
- Opérateurs : C# permet via la surcharge d'opérateur, d'ajouter à une classe les opérations mathématiques de base.

Les modificateurs d'accès suivants peuvent s'appliquer aux membres ci-dessus :

- **public** : membre accessible depuis l'extérieur, y compris depuis des classes non dérivées
- **protected** : membre inaccessible depuis l'extérieur, exception faite des

classes dérivées

- **private** : membre inaccessible depuis l'extérieur, classe dérivées comprises ; modificateur par défaut
- **internal** : membre visible uniquement à l'intérieur de l'unité de compilation courante.

Ces modificateurs peuvent s'appliquer aux classes de la manière suivante :

- **public** : classe visible par n'importe quel programme d'un autre espace de nom
- **protected** : classe visible seulement par toutes les autres classes héritant de la classe conteneur de cette classe
- **private** : classe visible seulement par toutes les autres classes du même namespace où elle est définie.
- **internal** : classe visible seulement par toutes les autres classes du même assembly.
- **Abstract** : peut être associé à l'un des 3 modificateurs (public, protected et internal). Classe abstraite non instanciable. Aucun objet ne peut être créé.
- **Sans mot clé** : classe qualifiée **internal**. Si c'est une classe interne elle est alors qualifiée **private**.

2.4.2 Héritage

L'héritage consiste à définir une classe qui s'appuie sur une autre classe en termes de données et de comportements. En outre, la classe dérivée peut s'utiliser à la place de la classe de base.

La syntaxe de déclaration est la suivante :

```
class ClasseDérivée : ClasseBase  
{    ...}
```

Par exemple :

```
class Personne  
{  
    private String nom;  
    private int age;  
    public int nbre_enf;  
    public Personne(String name, int n)  
        { nom=name; age=n;}  
}  
classe Eleve: Personne
```

```
{  
    private String ecole;  
    public String classe;  
    public Eleve(String n, int a, String e): base(n, a)  
    { ecole=e;}  
}
```

- Interfaces multiples

C# ne gère pas l'héritage multiple basée sur la dérivation. Par contre, l'on peut effectuer de l'héritage multiple avec des interfaces. Par exemple :

```
//héritage multiple non correct  
class Toto { ...}  
class Tata { ... }  
class Tototata : Toto, Tata {  
    ...  
}
```

```
//héritage multiple correct  
class Toto { ... }  
interface Itata { ... }  
interface Ititi { ... }  
class TotoTataTiti : Toto, Itata, Ititi{  
    ...  
}
```

- Classes scellées

Le modificateur ***sealed*** interdit à une classe de servir de classe de base (donc non héritable).

2.4.3 Les méthodes

Les méthodes (fonctions ou procédures) sont les codes opérant sur les données de l'objet.

- La méthode Main

Une application C# doit contenir une méthode Main, publique et statique, qui sert de point d'entrée. Main est généralement placée dans une classe ad hoc (qui convient). Par exemple :

```
class Etudiant {  
    private int etudiantId ;  
}  
class Prog {  
    static public void Main() {  
        Etudiant e = new Etudiant() ;  
    }  
}
```

Main peut prendre en arguments un tableau de chaînes de caractères, de sorte qu'à l'exécution du programme en ligne de commande l'on puisse passer des arguments. Main peut retourner une valeur entière qui servira de code erreur pour l'application appelante ou le fichier batch.

- Constructeur

C'est une méthode spéciale, exécutée chaque fois qu'il y a création d'une instance. Un constructeur garantit que l'objet sera correctement initialisé avant toute utilisation. Les constructeurs portent les noms des classes associées, par exemple :

```
using System ;  
class ConstructeurApp {  
    ConstructeurApp(){  
        Console.WriteLine("je suis un constructeur") ;  
    }  
    ...  
}
```

Un constructeur ne retourne jamais de valeur ; un objet s'instancie de la manière suivante :

<classe> <nom_objet> = new <classe>(arguments du constructeur)

- Paramètres REF et OUT des méthodes

Une méthode n'a qu'une seule valeur de retour, alors que parfois il est nécessaire de retourner plusieurs valeurs au programme appelant ; dans ce cas on utilise **ref** ou **out** devant les arguments de la méthode.

La différence entre **ref** et **out** est qu'avec **ref** il est nécessaire d'initialiser les arguments.

- Paramètres en nombre variable

L'on peut définir une méthode dont le nombre de paramètre est variable à l'aide du mot-clé **params**. Par exemple : *public double somme (params double[] liste)*

- Méthode statique

Une méthode statique existe au niveau global de la classe, pas au niveau d'une instance spécifique ; l'on n'a pas besoin d'instancier la classe pour pouvoir utiliser la méthode. Pour définir une méthode statique, on utilise le mot-clé **static** dans la définition ; on appelle une méthode statique par la syntaxe **Classe.Méthode** et non **Objet.Méthode**. Une méthode statique ne peut accéder qu'aux membres statiques de la classe.

- Méthode et classe abstraite

Le mot clé **abstract** est utilisé pour représenter une classe ou méthode abstraite.

Une méthode déclarée en abstract dans une classe mère :

- N'a pas de corps de méthode.
- N'est pas exécutable
- Doit obligatoirement être redéfinie dans une classe fille

Une méthode **abstraite** n'est qu'une **signature** de méthode sans implémentation dans la classe.

Une classe abstraite a les contraintes de définition suivantes :

- Si une classe contient au moins une méthode abstraite, elle doit être déclarée en classe abstract elle-même.
- Une classe **abstract** ne peut pas être instanciée directement, seule une classe dérivée (sous-classe) qui redéfinit obligatoirement toutes les méthodes **abstract** de la classe mère peut être instanciée.
- une classe dérivée qui redéfinit toutes les méthodes **abstract** de la classe mère sauf une (ou plus d'une) ne peut pas être instanciée et subit la même règle que la classe mère : elle contient au moins une méthode abstraite donc elle est aussi une classe abstraite et doit donc être déclarée en **abstract**.

N.B. : - Une classe **abstract** peut contenir des méthodes non abstraites et donc implantées dans la classe.

- Une classe **abstract** peut même ne pas contenir du tout de méthodes abstraites, dans ce cas une classe fille n'a pas la nécessité de redéfinir les méthodes de la classe mère pour être instanciée.

- Lorsqu'une classe est déclarée en **abstract** et que toutes ses méthodes sont déclarées en **abstract**, on appelle en C# une telle classe une **Interface**.

2.4.4 Le polymorphisme

Le polymorphisme est la capacité d'une hiérarchie de classes à fournir différentes implémentations de méthodes portant le même nom et par corollaire la capacité qu'ont les objets enfants de modifier les comportements hérités de leur parents. L'on a plusieurs types de polymorphisme :

- Polymorphisme d'objet

C'est une interchangeabilité entre variables d'objets de classes de la même hiérarchie sous certaines conditions. Nous avons deux cas de figure :
Soit une classe Mere et une classe Fille dérivée de la classe Mere

//polymorphisme d'objet implicite

```
Mere x ;  
Fille y = new Fille();  
x = y ;
```

//polymorphisme d'objet explicite : transtypage

```
Mere x ;  
Fille y ;  
Fille z = new Fille();  
x = z ; // x pointe sur un objet de type Fille  
y = (Fille) x ; //transtypage et affectation de référence
```

//affectation non acceptée

```
Mere x ;  
Fille y;  
x = new Mere(); //instantation dans le type initial  
y = (Fille)x ; ← erreur lors de l'exécution //affectation acceptée statiquement
```

//affectation après test du type

```
Mere x ;  
Fille y;  
x = new Mere();//instantation dans le type initial  
if (x is Fille) //test d'appartenance de l'objet référencé par x à la bonne  
classe
```

$y = (Fille)x ;$

- Polymorphisme de méthode

Lorsqu'une classe enfant hérite d'une classe mère, des méthodes supplémentaires nouvelles peuvent être implémentées dans la classe enfant mais aussi des méthodes des parents peuvent être substituées pour obtenir des implémentations différentes. On parle de polymorphisme de méthode. Il se présente sous deux formes :

Le polymorphisme statique ou la surcharge de méthode consiste dans le fait qu'une classe peut disposer de plusieurs méthodes ayant le même nom, mais avec des paramètres formels différents ou éventuellement un type de retour différent. On appelle signature d'une méthode l'entête de la méthode avec ses paramètres formels et leur type.

Le polymorphisme dynamique ou la **redéfinition** de méthode ou encore la surcharge héritée est mise en œuvre lors de l'héritage d'une classe mère vers une classe fille dans le cas d'une méthode ayant la même signature dans les deux classes. La redéfinition de méthode peut être selon le cas précoce ou tardive.

Si le compilateur effectue la liaison du code de la méthode immédiatement lors de la compilation la liaison est dite statique ou précoce. Dans ce cas la redéfinition de la méthode dans la classe dérivée n'est pas prise en compte ; c'est toujours la méthode définie dans la classe mère qui est visible (l'autre étant masquée).

Si le code est lié lors de l'exécution la liaison est dite dynamique ou tardive. Dans ce cas le compilateur recherche la bonne méthode. En C#, on utilise les qualificatifs **virtual** pour la méthode dans la classe mère et **override** pour la méthode dans la classe fille pour effectuer la liaison dynamique.

Par exemple :

```
//Exemple de liaison précoce
using System;
class Employe{
    public Employe (string nom) {
        this.Name = nom ;
    }
    protected string Name ;
    public string name {
        get {return this.Name ;}
    }
    public void CalculPaie() {
        Console.WriteLine("appel de Employe.CalculPaie pour ",name) ;
    }
}
```

```
    }  
}  
  
class EmployeCDD : Employe {  
    public EmployeCDD(string name) : base(name) {}  
    public new void CalculPaie() {  
        Console.WriteLine("appel de EmployeCDD.CalculPaie pour ", name) ;  
    }  
}  
  
class EmployeCDI : Employe {  
    public EmployeCDI(string name) : base(name) {}  
    public new void CalculPaie() {  
        Console.WriteLine("appel de EmployeCDI.CalculPaie pour ", name) ;  
    }  
}  
  
class TestAppl {  
    protected Employe[] employes;  
    public void EmployesCharges() {  
        employes = new Employe[2];  
        employes[0] = new EmployeCDD("Assalé");  
        employes[1] = new EmployeCDI("Adjé");  
    }  
    public void AffichagePaie() {  
        foreach(Employe emp in employes) {  
            emp.CalculPaie();  
        }  
    }  
    public static void Main() {  
        TestAppl test = new TestAppl();  
        test.EmployesCharges();  
        test.AffichagePaie();  
    }  
}  
  
//à l'exécution de l'application on aura  
appel de Employe.CalculPaie pour Assalé  
appel de Employe.CalculPaie pour Adjé  
  
//exemple de liaison tardive  
using System;  
class Employe{  
    public Employe (string nom) {  
        this.Name = nom ;
```

```
    }
    protected string Name ;
    public string name {
        get {return this.Name ;}
    }
    virtual public void CalculPaie() {
        Console.WriteLine("appel de Employe.CalculPaie pour ",name) ;
    }
}

class EmployeCDD : Employe {
    public EmployeCDD(string name) : base(name) {}
    override public void CalculPaie() {
        Console.WriteLine("appel de EmployeCDD.CalculPaie pour ", name) ;
    }
}

class EmployeCDI : Employe {
    public EmployeCDI(string name) : base(name) {}
    override public void CalculPaie() {
        Console.WriteLine("appel de EmployeCDI.CalculPaie pour ", name) ;
    }
}

class TestAppl {
    protected Employe[] employes;
    public void EmployesCharges() {
        employes = new Employe[2];
        employes[0] = new EmployeCDD("Assalé");
        employes[1] = new EmployeCDI("Adjé");
    }
    public void AffichagePaie() {
        foreach(Employe emp in employes) {
            emp.CalculPaie();
        }
    }
    public static void Main() {
        TestAppl test = new TestAppl();
        test.EmployesCharges();
        test.AffichagePaie();
    }
}

//à l'exécution de l'application on aura
appel de EmployeCDD.CalculPaie pour Assalé
```

appel de EmployeCDI.CalculPaie pour Adjé

2.4.5 Surcharge

En C#, l'on peut surcharger des méthodes comme des opérateurs.

- Surcharge de méthode

Elle consiste à créer plusieurs méthodes ayant le même nom, mais des arguments différents. Cette technique permet d'une part d'avoir une méthode dont le comportement varie selon les types de valeur qui lui sont passés et d'autre part d'instancier de plusieurs manières une classe (pour les constructeurs). Par exemple :

```
class Rectangle{
    private float L ;
    private float l ;
    Rectangle(float longueur, float largeur) //initialisation
    { L=longueur ; l=largeur ; }
    Rectangle() //autre initialisation possible
    {L=0 ; l=0 ; }
    private float perimetre()
    {return(2*(L+l)) ; }
    public float surface()
    {return (L*l); }
}
```

- Surcharge d'opérateur

Les opérateurs susceptibles d'être surchargés sont :

- Opérateur unaires : +, -, !, ~, ++, --, true et false ;
- Opérateurs binaires : +, -, *, /, %, &, |, ^, <<, >>, ==, !=, >, <, >= et <=

Les conditions suivantes doivent être respectées :

- Toute méthode sous-jacente à un opérateur surchargé doit être publique et statique.
- Techniquement, le type de retour peut être n'importe quel type, mais le plus souvent c'est le type pour lequel la méthode est définie (excepté pour les opérateurs true et false)
- Le nombre des arguments dépend de la nature de l'opérateur surchargé
- Pour un opérateur unaire, l'argument doit avoir le type de la classe ou structure englobante. Pour un opérateur binaire, le premier argument doit avoir le type de la classe ou structure et le second argument peut être de n'importe quel type.

La syntaxe de surcharge d'un opérateur *op* est :

```
public static type operator op (objet1 [, objet2])  
    { ...  
    }
```

2.4.6 Les membres simples

- Propriétés

Elles permettent d'accéder aux champs non publics comme s'il s'agissait de champs publics. Une propriété se compose d'une déclaration de champ et de méthodes (**getter** et **setter**) servant à accéder à la valeur du champ. Par exemple:

```
class Adresse {  
    protected string ville ;  
    protected string codePostal ;  
    public string CodePostal {  
        get {return codePostal ;}  
        set {  
            // Contrôle du Code Postal  
            codePostal = value ;  
            // calcul de la ville à partir du Code Postal  
        }  
    }  
}
```

Le paramètre mot clé **value** est utilisé pour recevoir la valeur à affecter.

Attention, ne pas confondre le champ `Adresse.codePostal` et la propriété `Adresse.CodePostal`.

Un exemple d'utilisation est :

```
Adresse adr = new Adresse() ;  
adr.CodePostal = "5893" ;  
string lecode = adr.CodePostal ;
```

Pour ne pas permettre de modifier un champ propriété, il suffit de ne pas définir le setter.

Comme pour les méthodes, une propriété peut être dotée du modificateur **virtual** ou **override**. Ce qui permettra à une classe dérivée d'hériter des propriétés et de les redéfinir.

- Indexateurs

Les indexateurs ou « tableaux intelligents » permettent de manipuler les objets comme s'ils étaient des tableaux.

La définition d'un indexateur est proche de celle d'une propriété sauf que premièrement, l'indexateur prend un argument `index` ; deuxièmement on utilise le mot clé **this** car c'est la classe elle-même qui est traitée comme tableau.

Par exemple :

```
class MaClasse {  
    public object this [int index] {  
        get { //lecture des données idoines}  
        set { //modification des données}  
    }  
}
```

Les détails d'implémentation des données et des méthodes **get/set** sont indépendants de l'indexateur. L'indexateur permet d'instancier la classe avec une syntaxe du genre :

```
MaClasse cls = new MaClasse() ;  
cls[0] = unObjet ;  
Console.WriteLine("élément ", cls[0]) ;
```

2.5 Interfaces

- Une interface C# est un contrat, elle peut contenir des **propriétés**, des **méthodes**, des **événements** ou des **indexeurs**, mais **ne** doit contenir **aucun champ** ou **attribut**.
- Une interface **ne** peut **pas** contenir des méthodes déjà implémentées.
- Une interface ne contient que des signatures (**propriétés, méthodes**).
- **Tous les membres** d'une interface sont automatiquement **public**.
- Une interface est héritable.
- On peut construire une hiérarchie d'interfaces.
- Pour pouvoir construire un objet à partir d'une interface, il faut définir une classe non abstraite implémentant **tous** les membres de l'interface.

Déclaration d'interface

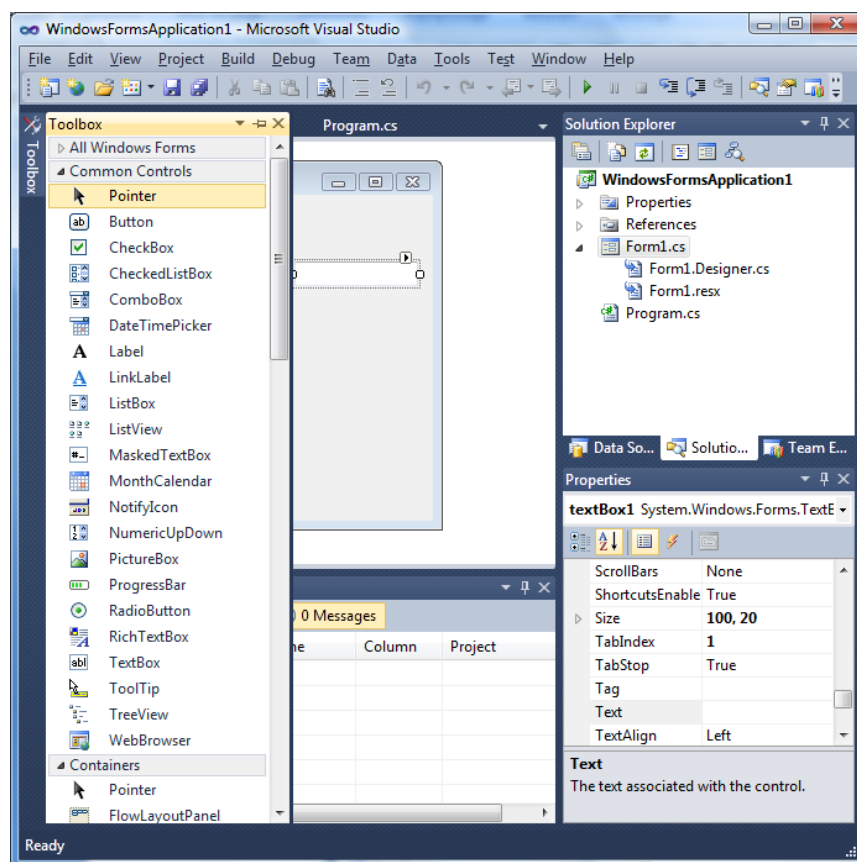
```
interface nomInterface {  
    //déclaration de méthode  
    type Methode1() ;
```

```
//déclaration de propriété  
type Propriete { get ;}  
//déclaration d'événement  
event typeEvenement Evenement ;  
//déclaration d'indexateur  
type this[int indx] {get ; set ;}  
}
```

2.6 Délégués et gestionnaires d'événements

3 Contrôles et Objets de base

La liste des contrôles utilisables est donnée par la boîte à outils située le plus souvent à gauche de la feuille de développement:



La liste des propriétés et des événements pour l'objet sélectionné est donnée par la fenêtre de propriétés située à droite et en bas.

3.1 Contrôles et objets usuels

Les propriétés communes à la plupart des contrôles sont :

- ✓ **Name** : le nom de l'objet ou contrôle. Il doit être unique sur une feuille. Il permet en programmation d'accéder aux propriétés et méthodes de l'objet par la notation : **<nomobjet>.<propriété> = valeur** ou **<nomobjet>.<méthode>**
- ✓ **TabIndex** : qui définit le rang d'accessibilité du contrôle sur la feuille
- ✓ **Enabled** : valeur à **True** définit que le contrôle est actif
- ✓ **Visible** : valeur à **True** définit que le contrôle est visible
- ✓ **Text** : définit le titre ou le contenu du contrôle

3.1.1 Contrôles Label, TextBox, Button, radioButton, ListBox, ComboBox

Le contrôle **Label** permet d'afficher de l'information à travers sa propriété **Text**, l'événement le plus important qui lui est associé est **TextChanged** : le code de cet événement est exécuté quand le contenu du **Label** change.

Le contrôle **TextBox** permet de saisir des informations, l'information saisie s'affiche via la propriété **Text**. Les événements les plus importants sont :

- ✓ **TextChanged** : exécuté quand le contenu change
- ✓ **KeyPress – KeyUp – KeyDown** : exécutés quand on appuie sur une touche – permet le contrôle de la saisie des caractères dans le **TextBox**
- ✓ **GotFocus – LostFocus** : exécutés quand l'on atteint le contrôle et quand l'on quitte le contrôle.

Les contrôles **ListBox** et **ComboBox** disposent de la propriété **Items** qui définit en mode conception la collection des éléments de la liste. En mode programmation, la méthode **Add** (en fin de liste) ou **Insert** (à une position spécifique) de l'objet **Items** permet d'ajouter un élément à la collection.

La propriété **Text** renvoie l'élément courant de la liste.

La propriété **SelectedIndex** renvoie ou définit l'index de l'élément sélectionné de la liste.

La propriété **SelectedText** pour un **ComboBox** renvoie ou définit le texte de l'élément courant.

Le **ListBox** dispose de la propriété **SelectionMode** qui permet de sélectionner : un seul élément dans la liste (**One**) – plusieurs éléments consécutifs (**MultiSimple**) – plusieurs éléments non consécutifs (**MultiExtended**).

L'événement le plus important est **SelectedIndexChanged** qui s'exécute quand on choisit un élément de la liste.

Les contrôles **RadioButton** présentent une suite de boutons radio dont un seul est sélectionnable. La propriété **Checked** à **True** spécifie que le bouton est sélectionné. La propriété **Text** définit le titre du bouton. L'événement le plus important est **CheckedChanged** qui s'exécute quand on a sélectionné un bouton.

Le contrôle **Button** spécifie un bouton de commande dont l'événement le plus important est **Click**. La propriété **Text** définit le titre du bouton. La propriété **Image** définit l'image à afficher sur le bouton. Pour qu'un bouton soit exécuté par la touche « Entrée », il faudrait que le nom du bouton soit choisi comme propriété **AcceptButton** de la feuille sur laquelle se trouve le bouton. De même, pour exécuter un bouton par la touche « Echap », il faudrait choisir le nom du bouton pour la propriété **CancelButton** de la feuille.

3.1.2 Les paramètres des événements

Un gestionnaire d'événements est une méthode qui est liée à un événement. Quand l'événement est déclenché, habituellement en réponse à un message du sous-système Windows, comme une touche appuyée ou un clic de souris, le code à l'intérieur du gestionnaire d'événements est exécuté.

Chaque gestionnaire d'événements liste deux paramètres qui sont utilisés dans le **handle**. Les gestionnaires d'événements de souris reçoivent un argument de type **MouseEventArgs**, les gestionnaires d'événements de touches appuyées reçoivent un argument de type **KeyPressEventArgs** et les autres gestionnaires d'événement un argument de type **EventArgs** ou **xxxEventArgs**.

Les propriétés de **MouseEventArgs** sont :

Propriété	Fonction
Button	Indique quel bouton de souris a été appuyé
Clicks	Indique combien de fois l'utilisateur a appuyé sur le bouton de souris ou l'a relâché
Delta	Renvoie un nombre signé indiquant le nombre de crans lors de la rotation de la roulette de la souris
X	Renvoie la coordonnée dans l'axe des X d'un clic de souris
Y	Renvoie la coordonnée dans l'axe des Y d'un clic de souris

Les propriétés de **KeyPressEventArgs** sont :

Propriété	Fonction
Handle	Renvoie une valeur indiquant si l'événement KeyPress a été géré
KeyChar	Renvoie la valeur KeyChar (caractère appuyé) qui correspond à la touche appuyée.

3.1.3 L'objet MessageBox

Il permet d'afficher une boîte de message à l'aide de sa méthode Show(). Par exemple : **MessageBox.Show(texte, titre, lesboutons, licône, leBoutonParDéfaut).**

lesboutons peut prendre les valeurs suivantes : **MessageBoxButtons.{AbortRetryIgnore | Ok | OkCancel | RetryCancel | YesNO | YesNoCancel}.**

licône peut prendre les valeur suivantes : **MessageBoxIcon.{Asterisk | Error | Hand | Information | None | Exclamation | Question | Stop | Warning}.**

leBoutonParDéfaut peut être : **MessageBoxDefaultButton.{Button1 | Button2 | Button3}**

3.1.4 Le contrôle Timer

Le contrôle **Timer** dispose de la propriété **Interval** qui spécifie en millisecondes le laps de temps entre deux déclenchements du timer (exécution de l'événement **timer_tick**).

L'objet **DateTime** par sa propriété **Now** renvoie la date et l'heure courante, que l'on peut convertir en chaîne à l'aide du format : **"hh:mm:ss"** pour retourner l'heure au format 12 heures et **"HH:mm:ss"** au format 24 heures.

3.1.5 Le contrôle PictureBox

Permet d'insérer les formats d'images suivants : JPEG – GIF WMF – BMP.

L'insertion d'image en mode conception s'effectue à l'aide de la propriété **Image**. En mode programmation on utilise la méthode **FromFile** des objets de la classe **Image** de la manière suivante : **PictureBox1.Image = image.FromFile(fichier_image)**

La Classe **Image** se trouve dans **System.Drawing**.

On peut charger une image en définissant sa propriété **picture** à partir de celle d'un autre contrôle par simple égalité.

Set pictureBox1.Picture = PictureBox2.Picture

La propriété **SizeMode** définit le placement et le dimensionnement de l'image dans le contrôle.

3.1.6 Contrôle ImageList

Sert à définir une collection d'images non visible à l'utilisateur. La propriété Images en mode conception permet d'ajouter des images.

En mode programmation, la propriété **images(i)** permet de récupérer l'image de rang i, par exemple : **PictureBox1.Image = ImageAlbum.Images[i]** ;

Pour ajouter des images dynamiquement à la collection on utilise la méthode **Add** de l'objet images de la collection par exemple :

ImageList1.images.Add(MonImage). Ici **MonImage** est un objet de la classe **System.Drawing.Image** qui récupère le fichier image par la méthode **FromFile** de la classe **Image**.

Les méthodes **remove(i)** et **clear()** respectivement supprime un ou toutes les images.

La collection d'images est exploitée par d'autres contrôles tels que les **ListView** et **TreeView**. La modification de l'image au sein du contrôle **ImageList** implique le changement automatique au niveau des deux autres contrôles. L'association s'effectue à l'aide de la propriété **ImageList** du contrôle. Certains contrôles disposent de la propriété ImageList pour être liée à un contrôle ImageList : Button – CheckBox – Label – ListView – RadioButton – TabControl – ToolBar – TreeView.

3.1.7 Les contrôles ListView et TreeView

Ce sont des contrôles qui disposent d'éléments Image et Texte.

Un **ListView** permet d'afficher des listes d'éléments selon 4 modes définis par la propriété **View** :

Mode grandes icônes – Mode petites icônes – Mode liste (liste sur une colonne) – Mode rapport (Liste personnalisable sur plusieurs colonnes).

Les principales propriétés sont :

Propriété	Description
Items	Définit la collection d'éléments
SelectedItems	Définit la collection d'éléments sélectionnés
View	Définit le mode d'affichage

LargeImageList	Définit le contrôle ImageList pourvoyeur d'images pour un affichage en mode grandes icônes
SmallImageList	Définit le contrôle ImageList pourvoyeur d'images pour un affichage en mode petites icônes
StateImageList	Définit le contrôle ImageList pourvoyeur d'images pour un affichage en état personnalisé (mode rapport)

Un élément d'un **ListView** est désigné par la classe **ListViewItem**.

Les méthodes **Remove** et **Add** de l'objet collection **Items** sont disponibles. La méthode **Add** utilise au moins deux paramètres : **ListView1.Items.Add(leTexte, leNumImage)** où **leTexte** est une chaîne de caractères représentant le libellé de l'image dans le **ListView** et **leNumImage** est le numéro de l'image dans un contrôle **ImageList** associé au **ListView**.

La propriété **MultiSelect** à **True** permet la sélection multiple dans le **ListView**.

Les éléments sélectionnés d'un **ListView** se trouve dans sa collection

SelectedItems. Un élément sélectionné dans le **ListView** peut donc être récupéré par **ListView1.SelectedItems[i].{Text | ImageIndex}** où **Text** est le texte associé à l'image dans le **ListView** et **ImageIndex** est l'index de l'image.

L'événement le plus important est **SelectedIndexChanged** qui s'exécute quand on choisit un élément de la liste.

Un **TreeView** propose une interface composée de divers nœuds hiérarchiques. Une exploitation classique est le parcours d'une arborescence de disque, de dossier ou de fichier.

L'arborescence est composé de nœuds dénommés nœuds parents, frères ou enfants selon leur position au sein de l'arborescence.

La propriété **SelectedNode** permet l'accès aux éléments sélectionnés.

L'événement **AfterSelect** gère aussi les éléments sélectionnés via la propriété **Node** du paramètre **e** qui est objet **TreeViewEventArgs** représentant la sélection.

L'objet de collection **Nodes** dispose des méthodes **Remove** et **Add** pour la suppression et l'ajout d'élément. Pour l'ajout d'élément, on définit d'abord un nouvel élément de la classe **TreeNode** dont le constructeur spécifie l'élément et l'index de l'image, on ajout ensuite cet élément au **TreeView**.

3.1.8 Les contrôles DomainUpDown et NumericUpDown

Le contrôle **DomainUpDown** se compose de zone de texte et d'une paire de boutons (des flèches). Il permet d'effectuer une sélection sur une liste. La propriété **Items** permet en mode conception de saisir du texte. En mode programmation deux méthodes servent à ajouter des éléments :

- Add pour l'ajout d'un élément en fin de liste, par exemple ***DomaineList.Items.Add(Texte);***;
- Insert pour l'ajout d'un élément à une position donnée ***DomaineList.items.Insert(position, Texte) ;***

Le contrôle ***NumericUpDown*** permet de définir une liste de numéro. Les propriétés ***Minimun*** fixe la valeur minimale, ***Maximum*** fixe la valeur maximale, la propriété ***Increment*** fixe le pas d'incrément et ***Value*** qui fixe une valeur initiale au lancement de l'application.

3.1.9 Le contrôle DateTimePicker

Permet à l'utilisateur de sélectionner un élément parmi une liste de dates et d'heures.

3.2 Quelques objets spécifiques

3.2.1 Les contrôles OpenFileDialog et SaveFileDialog

Sont dérivés de la classe de base ***CommonDialog***. Les propriétés communes sont :

- ✓ ***Filter*** définit le contenu de la liste des types de fichiers : exemple *Fichiers GIF | *.gif | Tous les fichiers | *.**
- ✓ ***InitialDirectory*** définit le répertoire par défaut
- ✓ ***Filename*** le nom du fichier

Ils disposent de la méthode ***ShowDialog()***.

OpenFileDialog.ShowDialog() affiche la boîte de dialogue Ouvrir de Windows et retourne une constante selon le bouton appuyé :

- ✓ ***DialogResult.OK*** : appui sur le bouton ***Ouvrir***
- ✓ ***DialogResult.CANCEL*** : appui sur le bouton ***Annuler***

La propriété ***Multiselect*** à ***True*** de ***OpenFileDialog*** permet des sélections multiples. En cas de sélection multiple, les différents fichiers se trouvent dans la collection ***FileNames*** de l'objet ***OpenFileDialog***.

L'objet ***SaveFileDialog*** dispose des propriétés suivantes :

- **CreatePrompt** = True 'message de confirmation si création de nouveau fichier
- **OverWritePrompt** = True 'message si le fichier existe déjà
- **DefaultExt** = 'txt' 'extension par défaut

La méthode **ShowDialog()** de l'objet **SaveFileDialog** retourne une constante selon le bouton appuyé :

- ✓ **DialogResult.OK** : appui sur le bouton **Enregistrer**
- ✓ **DialogResult.CANCEL** : appui sur le bouton **Annuler**

Cette méthode ouvre une boîte de dialogue permettant à l'utilisateur de choisir un nom et un chemin de fichier, c'est au programmeur d'écrire le code enregistrant les fichiers.

3.2.2 Le contrôle RichTextBox

Il autorise la saisie de textes, l'insertion d'images et de liens hyperrextes au sein d'un document avec gestion d'attributs de mise en forme (police, couleur, ...). Il permet la mise en œuvre d'un traitement de texte comme WordPad.

Pour mettre en forme un texte sélectionné, on procède de la manière suivante :

- ✓ Pour appliquer une couleur : **RichTextBox1.SelectionColor** = **Color.{Red | blue | ...}**
- ✓ Pour spécifier la police, la taille et le style : **RichTextBox1.SelectionFont** = **New Font (police, taille, style)** où police peut être "**Times New Roman**", "**Tahoma**", "**Algerian**", etc. taille un nombre entier et style de la forme **FontStyle.{Bold | Italic | Regular | Strikeout | Underline}**

Il dispose des méthodes :

- **SaveFile(nomfichier, mode)** : enregistre le texte dans le fichier spécifié ; le mode est soit **RichTextBoxStreamType.RichText** pour sauvegarder le texte enrichi, soit **RichTextBoxStreamType.PlainText** pour le texte brut.
- **Loadfile(nomfichier, mode)** permet d'ouvrir un fichier.

Ces méthodes sont utilisées conjointement avec un **SaveFileDialog** et **OpenFileDialog**.

Pour rechercher du texte, on utilise la méthode **Find** qui retourne l'emplacement d'index du premier caractère du texte recherché et le met en surbrillance (à moins de spécifier le contraire), sinon la valeur -1 est retournée. La syntaxe est : **RichTextBox1.Find(TexteRecherché, PositionDébut, PositionFin, Option)** où **PositionDébut** et **PositionFin** sont des entiers et **Option** peut prendre les valeurs suivantes : **RichTextBoxFinds.{MatchCase | NoHighlighth | None | Reverse | WholeWord}**

On peut copier et déplacer du texte sélectionné à l'aide des méthodes **Copy**, **Cut** et **Paste**. De même, on peut tester si certaines mises en forme ont été appliquées au texte sélectionné à l'aide des propriétés :

SelectionFont.Underline – **SelectionFont.StrikeOut** – **SelectionFont.Italic** – **SelectionFont.Bold** – **SelectionColor.<nomcouleur>**. On peut récupérer la taille, le nom de la police et la couleur du texte sélectionné à l'aide des propriétés : **SelectionFont.Size** – **SelectionFont.Name** – **SelectionColor.toString**.

Le point de départ du texte sélectionné est donné par la propriété **SelectionStart** et le nombre de caractères sélectionnés par **SelectionLength**.

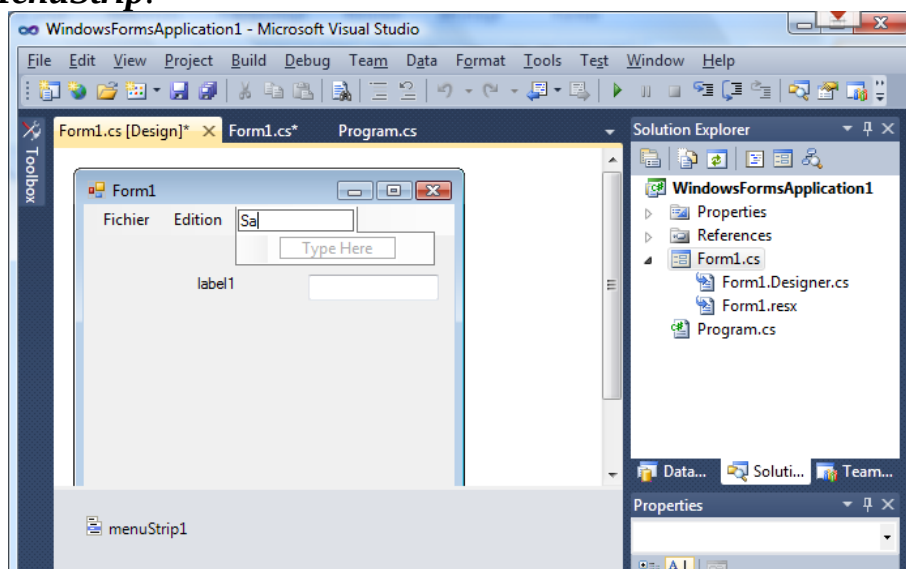
Les méthodes **Undo** et **Redo** sont disponibles pour les dernières modifications. Comme pour les **TextBox**, il y a une collection **Lines[]** qui contient chacune des lignes. La propriété **ReadOnly** à **True** permet d'interdire la modification.

3.2.3 L'objet Menu

La définition d'un menu pour un formulaire s'effectue par l'utilisation d'un objet **MainMenu** ou **MenuStrip**.

On saisit les éléments du menu dans la zone « Tapez ici ». En saisissant le texte du menu, une zone « Tapez ici » apparaît en bas pour définir les sous menus où à droite pour définir d'autres menus.

Pour créer un menu contextuel, on utilise un objet **ContextMenu** ou **ContextMenuStrip**.



3.2.4 Applications MDI

Une application **MDI** comprend un unique formulaire parent conteneur et une collection de fenêtres enfants **MDI**. On crée un **MDI** parent en transformant un formulaire existant ; on initialise sa propriété **IsMDIContainer** à **True**.

Pour que le formulaire enfant s'affiche à l'intérieur de la feuille **MDI**, on affecte à sa propriété **MdiParent** le nom de la feuille **MDI**.

Pour afficher un formulaire ou feuille on procède comme suit :

1) On déclare une instance du formulaire

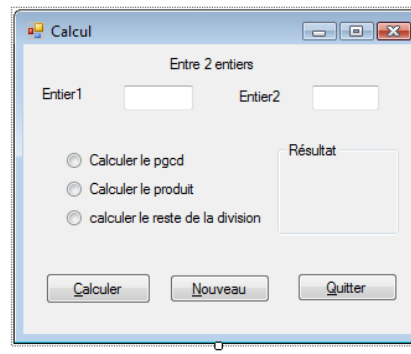
2) Ensuite on affiche le formulaire par : **<nom_instanceformulaire>.show()**

Pour fermer une feuille on utilise l'instruction **close()**, pour quitter une application on utilise l'instruction **Application.Exit()**.

4 Exercices

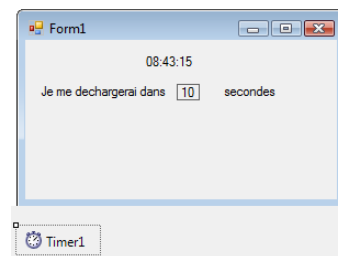
TP1 :

Réaliser cette application



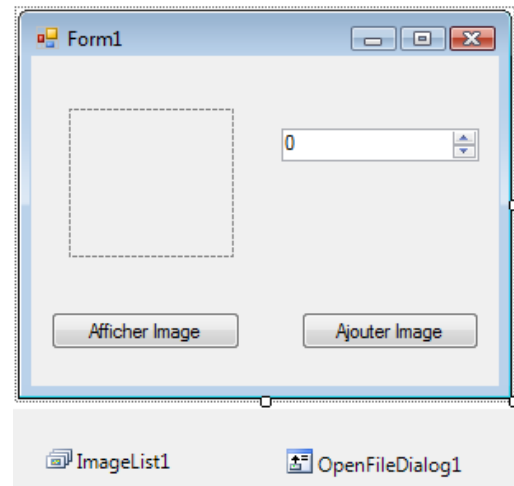
TP 2 :

Afficher l'heure courante à l'aide du contrôle Timer et d'un contrôle Label. Programmer le contrôle Timer de sorte que la feuille se décharge au bout d'un certain nombre de secondes par affichage d'un compte à rebours.



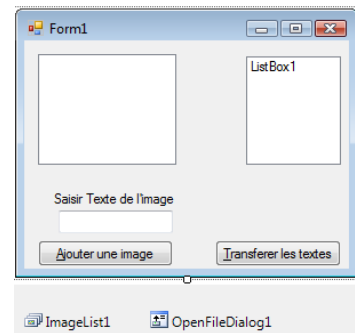
TP 3 :

- Remplir un objet ImageList avec des fichiers images ; on utilisera un bouton de commande « Ajouter Image » pour insérer l'image dans la liste d'images.
- Utiliser un objet NumericUpDown pour choisir un nombre et un bouton « Afficher Image » pour afficher l'image dont le rang dans la liste d'images correspond au nombre choisi.



TP 4 :

- Remplir un objet ListView avec un objet ImageList et un objet TextBox pour saisir le texte de l'élément.
- Transférer le texte des éléments sélectionnés dans un objet ListBox.



TP 5 :

Réaliser un petit éditeur de texte à l'aide d'un RichTextBox et des contrôles OpenFileDialog et SaveFileDialog